1. Descripción general del proyecto

Una empresa comercial cuenta con varias sedes de venta y un centro de distribución. Cada sede maneja productos, inventarios, pedidos, compras y transferencias. La organización necesita consultar sus datos localmente y obtener reportes consolidados sin perder la separación de responsabilidades entre sus áreas.

El proyecto consiste en diseñar e implementar en PostgreSQL una solución de datos basada en una ARQUITECTURA DE MICROSERVICIOS. Cada microservicio será responsable de un conjunto definido de entidades y operaciones; la información operativa se fragmentará por sede cuando corresponda y se integrará mediante consultas y vistas SQL para obtener informes institucionales.

El trabajo se centrará en el modelo Entidad–Relación, la transformación al modelo relacional, la normalización, las restricciones de integridad, los roles, las vistas, la distribución de los datos y las consultas SQL avanzadas. NO se exige programar microservicios, construir API REST, utilizar Docker ni desarrollar interfaces web o móviles. La arquitectura deberá quedar diseñada y representada; su capa de datos sí deberá implementarse y probarse.

2. Alcance y condiciones mínimas

- Definir al menos cuatro sedes, incluida una sede de distribución, y al menos cuatro bodegas o almacenes asociados.
- Representar catálogos de productos, categorías, proveedores, clientes y personal, además de compras, ventas, pagos, inventarios, transferencias y devoluciones.
- Organizar los módulos de datos de acuerdo con los microservicios especificados en el apartado de arquitectura, con límites claros de responsabilidad.
- Aplicar al menos una estrategia de fragmentación horizontal por sede a dos conjuntos operacionales, por ejemplo inventarios y pedidos de venta.
- Permitir tanto consultas individuales por sede como consultas corporativas que integren información de los distintos fragmentos.
- Crear datos de prueba que permitan responder las veinte consultas solicitadas y demostrar restricciones, roles y vistas en PostgreSQL.

3. Estructura sugerida de tablas y pertenencia a microservicios

La siguiente propuesta es una base de referencia y no un diseño definitivo. Los estudiantes pueden reorganizar o ampliar las entidades, siempre que mantengan los requerimientos y documenten la pertenencia de cada tabla a un microservicio. Deben definir claves primarias, claves foráneas aplicables, tipos de datos, restricciones y cardinalidades.

Microservicio de Catálogo y Terceros

Sedes = (id_sede, nombre, ciudad, direccion, tipo_sede, estado)

Almacenes = (id_almacen, id_sede, nombre, tipo_almacen, estado)

Categorias = (id_categoria, nombre, descripcion, estado)

Productos = (id_producto, sku, nombre, id_categoria, unidad_medida, precio_lista, estado)

Proveedores = (id_proveedor, identificacion, razon_social, ciudad, correo, telefono, estado)

Producto_Proveedor = (id_producto, id_proveedor, costo_referencia, plazo_entrega_dias)

Clientes = (id_cliente, tipo_documento, num_documento, nombre_completo, correo, telefono, estado)

Microservicio de Compras y Abastecimiento

Ordenes_Compra = (id_orden_compra, id_proveedor, id_almacen_destino, id_empleado, fecha_orden, fecha_estimada, estado)

Detalle_Orden_Compra = (id_orden_compra, id_producto, cantidad_solicitada, costo_unitario)

Recepciones_Compra = (id_recepcion, id_orden_compra, fecha_recepcion, estado)

Detalle_Recepcion = (id_recepcion, id_producto, cantidad_recibida)

Microservicio de Inventarios y Transferencias

Inventarios = (id_sede, id_almacen, id_producto, existencia_actual, stock_minimo, fecha_actualizacion)

Transferencias = (id_transferencia, id_almacen_origen, id_almacen_destino, fecha_solicitud, fecha_despacho, fecha_recepcion, estado)

Detalle_Transferencia = (id_transferencia, id_producto, cantidad_enviada, cantidad_recibida)

Microservicio de Ventas y Pagos

Pedidos_Venta = (id_pedido, id_sede, id_almacen, id_cliente, id_empleado, fecha_pedido, estado, total_registrado)
Detalle_Pedido = (id_pedido, id_producto, cantidad, precio_unitario)
Pagos = (id_pago, id_pedido, fecha_pago, valor, medio_pago, estado)
Devoluciones = (id_devolucion, id_pedido, fecha_solicitud, motivo, estado)
Detalle_Devolucion = (id_devolucion, id_producto, cantidad, condicion_producto)

Microservicio de Identidad y Accesos

Empleados = (id_empleado, id_sede, num_documento, nombre_completo, cargo, correo, estado)
Usuarios = (id_usuario, id_empleado, nombre_usuario, password_hash, rol, estado, fecha_creacion)
Auditoria_Accesos = (id_auditoria, id_usuario, fecha_hora, accion, objeto_consultado)
Microservicio de Reportes: no requiere tablas de negocio adicionales; se representa mediante vistas y consultas consolidadas de solo lectura. No todas las relaciones entre microservicios tienen por qué materializarse como claves foráneas entre bases independientes; deberá identificarse cuáles se aseguran en PostgreSQL y cuáles se documentan mediante identificadores compartidos.

4. Requerimientos específicos del sistema

1. Administración por sede. Cada sede debe identificarse de forma única y estar asociada con sus almacenes, empleados e información operativa.
2. Microservicios. El diseño debe separar catálogo, compras, inventarios, ventas, identidad y reportes mediante responsabilidades y estructuras de datos definidas.
3. Catálogo. Cada producto debe tener un SKU único, una categoría y un estado. Un producto puede relacionarse con uno o varios proveedores.
4. Compras. Cada orden de compra se relaciona con un proveedor y contiene uno o varios productos, cantidades, costos y fechas.
5. Recepciones. Se deben poder registrar compras recibidas total o parcialmente, asociándolas a la orden y productos correspondientes.
6. Existencias. Se debe almacenar por sede y almacén la cantidad disponible registrada y el stock mínimo para cada producto.
7. Ventas. Cada pedido se asocia a una sede, cliente y empleado, y puede contener varios productos con cantidades y precios.
8. Pagos. Una venta puede asociarse a varios pagos; deben consultarse pagos confirmados y saldos pendientes.
9. Transferencias. Debe registrarse qué productos se trasladan entre almacenes, con origen, destino, fechas y cantidades.
10. Devoluciones. Deben almacenarse las devoluciones asociadas con pedidos existentes, los productos y el motivo.
11. Integridad. Se deben implementar PK, FK cuando apliquen, UNIQUE, NOT NULL y CHECK para cantidades, valores y estados.
12. Históricos. Los valores unitarios de una venta o compra deben conservarse en sus detalles para reportar correctamente periodos anteriores.
13. Permisos. Los roles del sistema deben tener privilegios diferenciados de consulta y modificación según sus funciones.
14. Vistas. Los reportes recurrentes y de consolidación deben poder consultarse mediante vistas SQL.
15. Distribución. Los datos de inventarios y pedidos deben presentarse fragmentados horizontalmente por sede, con una consulta consolidada.
16. SQL avanzado. Las consultas deben combinar relaciones, subconsultas, CTE, agrupaciones, operaciones de conjuntos y funciones de ventana.
17. Datos de prueba. Se deben incluir registros suficientes, variados y coherentes que permitan comprobar los resultados solicitados.
18. Arquitectura documentada. Cada entidad y vista debe tener un responsable lógico; deben explicarse las interacciones entre microservicios y las consultas de integración.

5. Consultas SQL obligatorias

Desarrollar y ejecutar las veinte consultas siguientes utilizando registros de prueba representativos. En el conjunto deben observarse JOIN, LEFT JOIN, NOT EXISTS, subconsultas correlacionadas, GROUP BY, HAVING, al menos cuatro CTE, al menos tres funciones de ventana y al menos dos operaciones de conjuntos (UNION, INTERSECT o EXCEPT). Se exige mostrar la instrucción SQL, el resultado y una breve interpretación. Las consultas consolidadas deben integrar datos de varias sedes.

1. Inventario por producto y sede. Mostrar producto, categoría, sede, almacén, existencia, stock mínimo y valorización estimada del inventario.

2. Alertas de reabastecimiento. Identificar productos con existencia igual a cero o inferior al stock mínimo en un almacén y ordenarlos por nivel de faltante.

3. Productos ausentes por sede. Encontrar los productos activos que no cuentan con registro de inventario en una o más sedes.

4. Ranking de productos por sede. Mostrar los cinco productos más vendidos de cada sede y periodo mediante DENSE_RANK() o ROW_NUMBER().

5. Ventas mensuales y variación. Calcular los valores vendidos por sede y mes; comparar con el mes anterior utilizando LAG() para estimar la variación.

6. Clientes con compras multisede. Identificar clientes con compras en al menos dos sedes, indicando sedes visitadas y total comprado.

7. Pedidos con pagos pendientes. Mostrar pedido, cliente, valor total, pagos confirmados acumulados y saldo por pagar.

8. Pedidos con diversidad de productos. Consultar pedidos con tres o más productos distintos y sumar las unidades y el importe registrado.

9. Pedidos antiguos sin completar. Identificar pedidos pendientes con antigüedad superior a un umbral definido respecto de la fecha consultada.

10. Demanda frente al inventario. Detectar detalles de pedidos pendientes cuya cantidad supera la existencia registrada del producto en su almacén.

11. Cumplimiento de proveedores. Calcular por proveedor el total de órdenes, las recibidas dentro del plazo y el promedio de días de retraso.

12. Compras pendientes de recibir. Comparar cantidades solicitadas y cantidades recibidas por orden y producto, incluyendo órdenes con recepción parcial.

13. Transferencias en tránsito. Listar traslados despachados y no recibidos con origen, destino, productos y días transcurridos.

14. Diferencias entre enviado y recibido. Encontrar transferencias con cantidades recibidas distintas a las enviadas, por producto.

15. Redistribución de productos. Buscar pares de sedes con excedente sobre el stock mínimo en una y faltante en otra para el mismo producto.

16. Productos de baja rotación. Encontrar productos con existencia positiva que no han registrado ventas confirmadas durante los últimos noventa días.

17. Devoluciones por producto y categoría. Calcular cantidades vendidas, devueltas y porcentaje de devolución en un periodo.

18. Comparación de surtido entre sedes. Mediante EXCEPT o INTERSECT, identificar los productos que existen en una sede y no en otra, o los comunes a ambas.

19. Clasificación de vendedores. Mostrar por sede el total vendido y el puesto de cada empleado utilizando RANK() o ROW_NUMBER().

20. Consolidado corporativo. Generar un informe por sede con productos activos, unidades en inventario, ventas, pedidos pendientes, compras y transferencias. Utilizar CTE o subconsultas agregadas para evitar conteos duplicados.

6. Vistas SQL obligatorias

Crear al menos diez vistas; algunas serán de uso local y otras de consulta consolidada. Deben tener nombres claros, ejecutarse sin errores y mostrar datos coherentes con las tablas fuente.

1. vista_inventario_por_sede
2. vista_alertas_reabastecimiento
3. vista_productos_sin_inventario

4. vista_ventas_mensuales_por_sede
5. vista_pedidos_pendientes
6. vista_saldos_pendientes
7. vista_compras_pendientes
8. vista_transferencias_en_transito
9. vista_devoluciones_por_producto
10. vista_reporte_corporativo

Las vistas corporativas se implementarán en el contexto de reportes, y deberán quedar identificados los fragmentos o microservicios de donde proceden los datos. No es necesario implementar mecanismos de sincronización en tiempo real.

7. Lineamientos de integración entre microservicios

El proyecto adopta una arquitectura de microservicios como decisión obligatoria de diseño. Cada equipo deberá:

- Definir y dibujar los microservicios, sus responsabilidades, las entidades que administran y sus dependencias de información.
- Crear un esquema PostgreSQL propio para cada microservicio de datos. El esquema reportes contendrá únicamente vistas y consultas de integración.
- Mantener separadas las operaciones sobre datos: un rol de un microservicio no deberá tener permiso de modificación sobre tablas administradas por otro.
- Utilizar identificadores compartidos y documentar las relaciones entre entidades pertenecientes a servicios distintos.
- Demostrar las consultas de integración mediante vistas SQL de solo lectura sobre los esquemas o fragmentos de sede.

Una única instancia de PostgreSQL con esquemas separados es una simulación académica de la arquitectura de datos de microservicios; no equivale por sí sola a desplegar microservicios independientes. Esta distinción deberá explicarse expresamente en el informe.

8. Roles, usuarios y permisos

1. Administrador. Crea esquemas y usuarios, administra permisos y consulta toda la información.

2. Gerencia. Consulta reportes consolidados y resultados de las sedes sin modificar información operativa.

3. Responsable de inventario. Consulta y actualiza datos de inventario y transferencias de las sedes autorizadas.

4. Responsable de compras. Gestiona órdenes de compra y recepciones, según las autorizaciones establecidas.

5. Vendedor. Registra y consulta pedidos y pagos de su sede, sin gestionar tablas de otros servicios.

6. Auditor. Consulta información y reportes autorizados sin permisos de modificación.

Entregar scripts CREATE ROLE, GRANT y REVOKE, junto con una matriz de permisos por rol, esquema y operación (SELECT, INSERT, UPDATE, DELETE). Demostrar por lo menos dos accesos permitidos y dos denegados. No se exige implementar autenticación web, políticas RLS ni otros mecanismos de seguridad fuera de los temas tratados en clase.

9. Arquitectura de microservicios y estrategia de distribución

9.1 Arquitectura requerida

La solución deberá modelarse mediante los siguientes microservicios: (1) Catálogo y Terceros; (2) Compras y Abastecimiento; (3) Inventarios y Transferencias; (4) Ventas y Pagos; (5) Identidad y Accesos; y (6) Reportes Corporativos. Es posible proponer una variante con justificación, siempre que exista separación por responsabilidades y una capa de consultas institucionales.

Cada microservicio es propietario lógico de sus datos. El diseño del diagrama debe indicar límites, entidades principales, consultas locales, vínculos de información entre servicios y la procedencia de los reportes. La comunicación por API u otros mecanismos se representará solo conceptualmente. No hay obligación de desplegar infraestructura adicional.

9.2 Distribución lógica obligatoria

La implementación mínima aceptada usará una instancia de PostgreSQL con esquemas por microservicio (por ejemplo, catalogo, compras, inventarios, ventas, identidad y reportes). Dentro de inventarios y ventas se crearán fragmentos horizontales por sede para dos conjuntos operacionales elegidos, mediante tablas físicas separadas o una estrategia de particionamiento equivalente que pueda demostrarse. Un ejemplo es inventarios.sede_norte_inventarios, inventarios.sede_sur_inventarios y una vista UNION ALL que los reúna.

Los estudiantes deberán documentar la condición de fragmentación, su cobertura de sedes, cómo se evita la duplicación de registros entre fragmentos, las claves empleadas y una vista unificada para reportes. No basta declarar un filtro WHERE como si fuera fragmentación física.

10. Datos de prueba y casos de consulta

Cargar al menos cuatro meses de registros para que las consultas produzcan resultados variados. Se proponen los siguientes mínimos:

- 4 sedes, incluida una de distribución, y 4 almacenes o más.
- 80 productos distribuidos en al menos 5 categorías.
- 10 proveedores, 100 clientes y 20 empleados.
- 300 pedidos de venta con sus detalles y distintos estados.
- 60 órdenes de compra y recepciones asociadas.
- 25 transferencias y 20 devoluciones, con sus respectivos detalles.

Los datos pueden ser sintéticos y generarse con scripts SQL, siempre que sean coherentes. Deben incluirse sedes con existencias distintas, productos sin ventas recientes, clientes multisede, proveedores con retrasos, pedidos con pagos parciales, compras incompletas y transferencias aún en tránsito.

11. Pruebas mínimas de verificación

Presentar una matriz breve con el SQL ejecutado, el resultado esperado, el resultado obtenido y su evidencia. Realizar al menos las siguientes verificaciones:

1. Crear un producto válido y consultar su registro.
2. Intentar duplicar un SKU y comprobar que se rechaza por una restricción UNIQUE.
3. Intentar registrar una cantidad negativa y verificar la restricción CHECK correspondiente.
4. Intentar referenciar una clave inexistente en una relación con FK local y comprobar el rechazo.
5. Consultar por separado los fragmentos de dos sedes y mostrar que contienen registros distintos.
6. Ejecutar la vista consolidada de inventarios y comprobar que reúne los fragmentos de las cuatro sedes.
7. Ejecutar una consulta avanzada basada en CTE y otra que utilice una función de ventana.
8. Comparar dos sedes mediante EXCEPT o INTERSECT, explicando los resultados.
9. Comprobar accesos permitidos y denegados de al menos dos roles.
10. Ejecutar el reporte corporativo sin duplicar ventas ni cantidades por efecto de relaciones uno a muchos.

No se solicitarán pruebas de concurrencia, demostraciones de bloqueo, reversión de transacciones, automatización mediante triggers ni conciliaciones complejas de movimientos de inventario.

12. Entregables

La entrega deberá contener una carpeta ordenada con los siguientes elementos:

1. Documento con descripción del problema, alcance, supuestos y reglas de negocio.
2. Diagrama de arquitectura de microservicios con responsables de cada grupo de datos e interacciones conceptuales.
3. Modelo conceptual Entidad–Relación con cardinalidades.
4. Modelo relacional, normalización hasta tercera forma normal y diccionario de datos.
5. Descripción de esquemas, servicios, fragmentos por sede y vistas de integración.
6. Scripts SQL de creación de esquemas, tablas, claves y restricciones.
7. Scripts de inserción o generación reproducible de datos de prueba.
8. Scripts de usuarios, roles, concesión y revocación de permisos.
9. Scripts de fragmentación de datos y vistas unificadas por sede.
10. Scripts de al menos diez vistas SQL.
11. Scripts de las veinte consultas avanzadas numeradas.

12. Resultados y evidencias de las consultas y de las pruebas de restricciones y permisos.
13. Análisis de resultados, interpretación de consultas y justificación de decisiones de distribución.
14. Manual de instalación y ejecución con PostgreSQL, pgAdmin o psql.
15. Conclusiones, limitaciones y posibles extensiones de la solución.